

Rapport de Conception

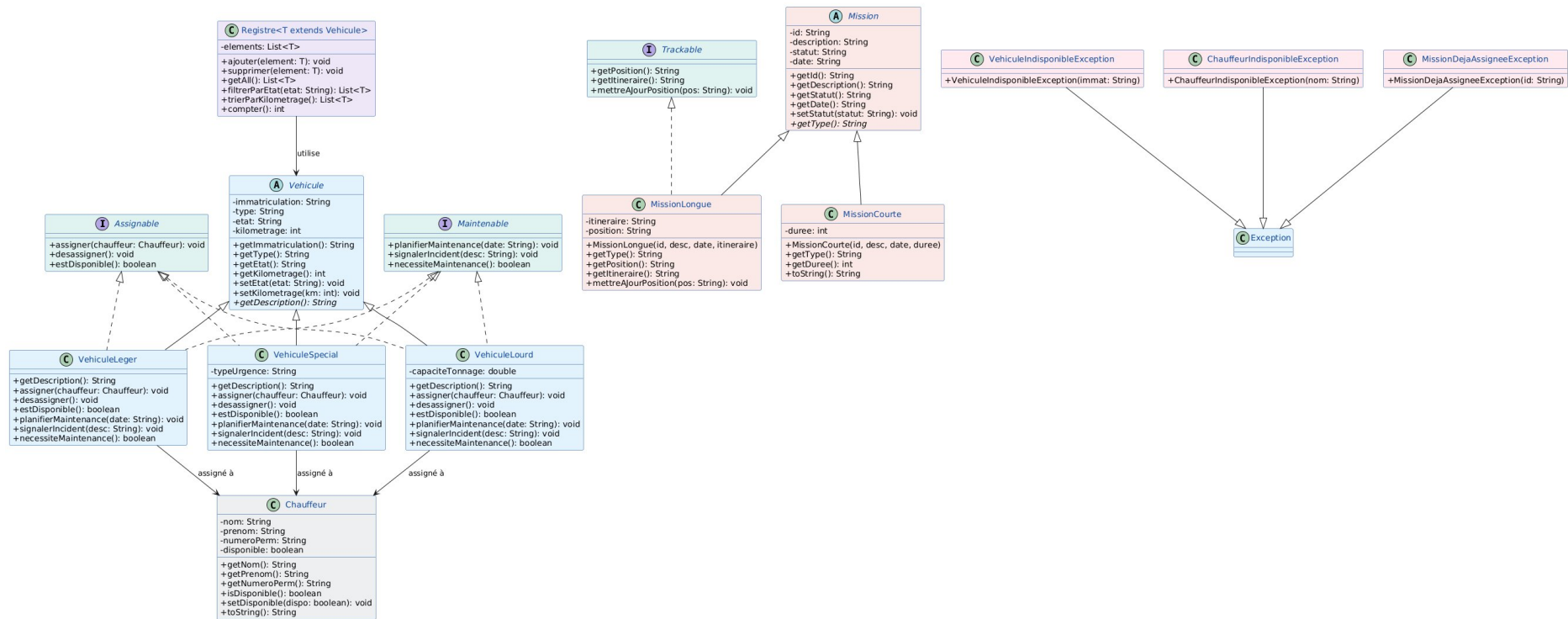
Gestionnaire de Flotte de Vehicules

Projet de groupe - POO Avancee

Membres du groupe
Guillaume CHAMMAH
Kyllian LEGROS
Killian LE CLAINCHE
Bruno DE CARVALHO LOPES

1. Diagramme de Classes UML

Le diagramme ci-dessous represente l'architecture complete du projet, avec les relations d'heritage, d'implementation et d'association.



2. Justification des Choix de Conception OO

2.1 Classes abstraites

Nous avons defini deux classes abstraites : Vehicule et Mission. Vehicule est abstraite car on ne cree jamais un simple Vehicule, mais toujours un VehiculeLeger, VehiculeLourd ou VehiculeSpecial. La methode getDescription() est abstraite car chaque type de vehicule decrit differemment ses caracteristiques. Les attributs communs (immatriculation, type, etat, kilometrage) sont centralises dans Vehicule pour eviter la duplication de code. Mission est abstraite pour la meme raison : on distingue une MissionCourte (avec duree) d'une MissionLongue (avec itineraire et position). La methode getType() est abstraite car chaque mission retourne un libelle different.

2.2 Interfaces multiples

Interface	Justification du choix
Assignable	Un vehicule peut etre assigne : c'est une capacite, pas une identite. Plusieurs types de vehicules partagent cette capacite sans lien d'heritage direct.
Maintenable	La maintenance est une responsabilite transversale separee de l'heritage pour respecter le principe de segregation des interfaces.
Trackable	Seules les MissionsLongues necessitent un suivi de position. L'interface permet d'ajouter cette capacite sans modifier la hierarchie de Mission.

2.3 Generics bornes

La classe Registre est un conteneur generique. La borne 'extends Vehicule' garantit que T dispose des methodes de Vehicule (getEtat(), getKilometrage()), ce qui permet d'ecrire des methodes generiques comme filtrerParEtat() et trierParKilometrage() sans cast. Ce choix evite la duplication d'une collection pour chaque type de vehicule.

2.4 Collections justifiees

Collection	Usage	Justification
List	Vehicules dans Registre	Ordre d'insertion conserve, acces par index
List	Liste des chauffeurs	Ordre d'insertion, iteration simple
List	Liste des missions	Ordre chronologique d'ajout
PriorityQueue	File vehicules speciaux	Priorite par type d'urgence
TreeMap	Agenda des missions	Tri naturel par date automatique
Map	Stats par etat	Association etat vers nombre de vehicules

2.5 Streams et Lambdas

Nous avons utilise l'API Stream Java 8+ dans GestionnaireFlotte pour tous les traitements. La methode `filtrerVehicules()` chaine plusieurs `filter()` pour combiner les criteres etat, type et kilometrage. La methode `getVehiculesParEtat()` utilise `collect(Collectors.groupingBy())` pour agreger les vehicules par etat. Les Comparator sont exprimes sous forme de lambdas pour le tri dynamique croissant et decroissant.

3. Difficultes Rencontrees et Solutions Adoptees

Difficulte	Solution adoptee
Gestion du classpath avec VS Code	VS Code ne trouvait pas les classes des autres packages. Resolution par compilation manuelle avec javac en specifiant tous les packages source.
Conflits Git lors des fusions	Plusieurs membres modifiaient des fichiers proches. Adoption d'une strategie de branches separees par membre et fusion sequentielle sur main.
Mise a jour dynamique de l'interface Swing	Les statistiques ne se mettaient pas a jour apres modification. Ajout d'un bouton Actualiser et utilisation de <code>SwingUtilities.invokeLater()</code> .
Exceptions dans l'interface Assignable	L'interface ne pouvait pas declarer d'exceptions checked. Gestion des exceptions dans <code>GestionnaireFlotte</code> plutot que dans l'implementation.

4. Ameliorations Envisagees avec Plus de Temps

Amelioration	Description
Base de donnees	Remplacer la persistance CSV par SQLite ou H2 pour une gestion plus robuste avec des requetes SQL pour le filtrage et la recherche multicriteres.
Tableau de bord visuel	Ajouter des graphiques (camembert, histogramme) avec JFreeChart pour rendre les statistiques plus lisibles et visuelles.
Historique par chauffeur	Ajouter un historique complet des missions par chauffeur avec calcul du total de kilometres et du taux de disponibilite.
Validation des formulaires	Validations en temps reel : format d'immatriculation, format de date, verification de doublons sur immatriculations et numeros de permis.
Architecture MVC stricte	Creer des controllers dedies (VehiculeController, ChauffeurController, MissionController) pour mieux respecter le principe de responsabilite unique.